

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Технологии и методы программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

«Локальная и веб-защита»

Выполнили:

Суханкулиев Мухаммет,
студент группы N3346



(подпись)

Проверил:

Ищенко Алексей Петрович,
преподаватель, ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Введение.....	3
1 Задание 1(а): Локальная программа.....	4
1.1 Задание.....	4
1.2 Механизмы защиты файлов в ОС Windows.....	4
1.3 Практическая реализация	5
1.4 Инструкция по применению.....	5
1.4.1 Первоначальная настройка	5
1.4.2 Использование	6
2 Задание 1(б): Веб-скрипт	7
2.1 Задание.....	7
2.2 Методы защиты контента на веб-страницах.....	7
2.3 Практическая реализация	7
2.4 Демонстрация работы	8
Заключение.....	9
Список использованных источников.....	10

ВВЕДЕНИЕ

Цель работы – разработка и исследование механизмов защиты данных на уровне файловой системы и веб-страниц.

Для достижения поставленной цели необходимо разработать:

- локальную программу для защиты файлов в каталоге с управлением доступом и паролем;
- веб-скрипт для ограничения копирования и сохранения контента страниц с возможностью печати и отключением по паролю.

1 ЗАДАНИЕ 1(А): ЛОКАЛЬНАЯ ПРОГРАММА

1.1 Задание

Разработать программу для ОС Windows, которая запрещает создание, копирование, удаление или переименование файлов с заданными именами или по маске. Список шаблонов должен храниться в текстовом файле `template.tbl`. Необходимо обеспечить защиту этого файла от несанкционированного доступа. Отключение защиты должно производиться по паролю, хеш которого хранится в файле `template.tbl`. Программа должна поддерживать режимы включения и выключения защиты.

1.2 Механизмы защиты файлов в ОС Windows

В операционных системах семейства Windows безопасность объектов файловой системы, таких как файлы и каталоги, регулируется с помощью **списков контроля доступа (Access Control Lists, ACL)**. Каждый защищаемый объект имеет связанный с ним **дескриптор безопасности (Security Descriptor)**, который содержит:

- **SID (Security Identifier)** владельца объекта.
- **DACL (Discretionary Access Control List)** – дискреционный – список контроля доступа. Он содержит набор **записей контроля доступа (Access Control Entries, ACE)**, которые определяют права (разрешение или запрет) для конкретных пользователей или групп.
- **SACL (System Access Control List)** – системный список, используемый для аудита доступа.

Ключевым для данной работы является DACL. Путем добавления в DACL файла **запрещающей записи (ACCESS_DENIED_ACE)** для группы «Все» (Everyone) можно заблокировать любые действия с файлом (**GENERIC_ALL**), включая чтение, запись, удаление и изменение прав. Такой запрет имеет высший приоритет и не может быть обойден даже пользователем с правами администратора.

Для хранения пароля используется **хеширование с солью**. Пароль не хранится в открытом виде. Вместо этого он конкатенируется со случайной строкой (солью), и от полученного результата вычисляется хеш-функция (например, SHA-256). Это защищает от атак по радужным таблицам.

1.3 Практическая реализация

Программа реализована на языке Python 3 с использованием библиотек `pywin32` для взаимодействия с Windows API и `watchdog` для мониторинга файловой системы. Архитектура программы модульная и объектно-ориентированная, что обеспечивает гибкость и расширяемость.

Основные компоненты:

- ***ConfigManager***: Класс для управления файлом конфигурации `template.tbl`. Отвечает за чтение, создание, проверку пароля (с использованием `hashlib.sha256` и соли) и парсинг шаблонов файлов.
- ***SecurityManager***: Ключевой класс, инкапсулирующий работу с Windows DACL. Метод `set_deny_all` получает дескриптор безопасности файла, модифицирует его DACL, добавляя или удаляя запрещающую запись для SID группы «Все», и применяет изменения.
- ***FileWatcher***: Класс, реализующий режим слежения. С помощью `watchdog` он отслеживает события создания (`on_created`) и перемещения/переименования (`on_moved`) файлов в реальном времени. Если имя файла соответствует одному из шаблонов, он немедленно вызывает ***SecurityManager*** для установки защиты.
- ***AppController***: Координирует работу всех модулей.
- ***ProtectorGUI***: Реализует графический пользовательский интерфейс на `tkinter`.

1.4 Инструкция по применению

Программа представляет собой единый исполняемый файл `protector.exe`, который может работать в двух режимах. Для работы требуются права администратора.

1.4.1 Первоначальная настройка

Поместите `protector.exe` в защищаемый каталог.

1. Запустите командную строку (`cmd.exe`) в этом каталоге.
2. Выполните команду: `protector.exe setup`.
3. Программа запросит создание и подтверждение пароля.
4. В каталоге будет создан файл `template.tbl`, в который необходимо внести маски защищаемых файлов (например, `*.docx`, `secret.txt`).

1.4.2 Использование

– **Графический интерфейс (GUI):** Запускается двойным щелчком по `protector.exe`. Позволяет наглядно видеть статус защиты и слежения, а также включать и выключать их с помощью кнопок.

– **Интерфейс командной строки (CLI):**

– **`protector.exe on`:** Включить защиту для файлов из `template.tbl`.

– **`protector.exe off`:** Отключить защиту (потребуется ввод пароля).

– **`protector.exe watch`:** Включить режим постоянного слежения и защиты (остановка по `Ctrl+C`).

– **`protector.exe status`:** Проверить текущий статус защиты.

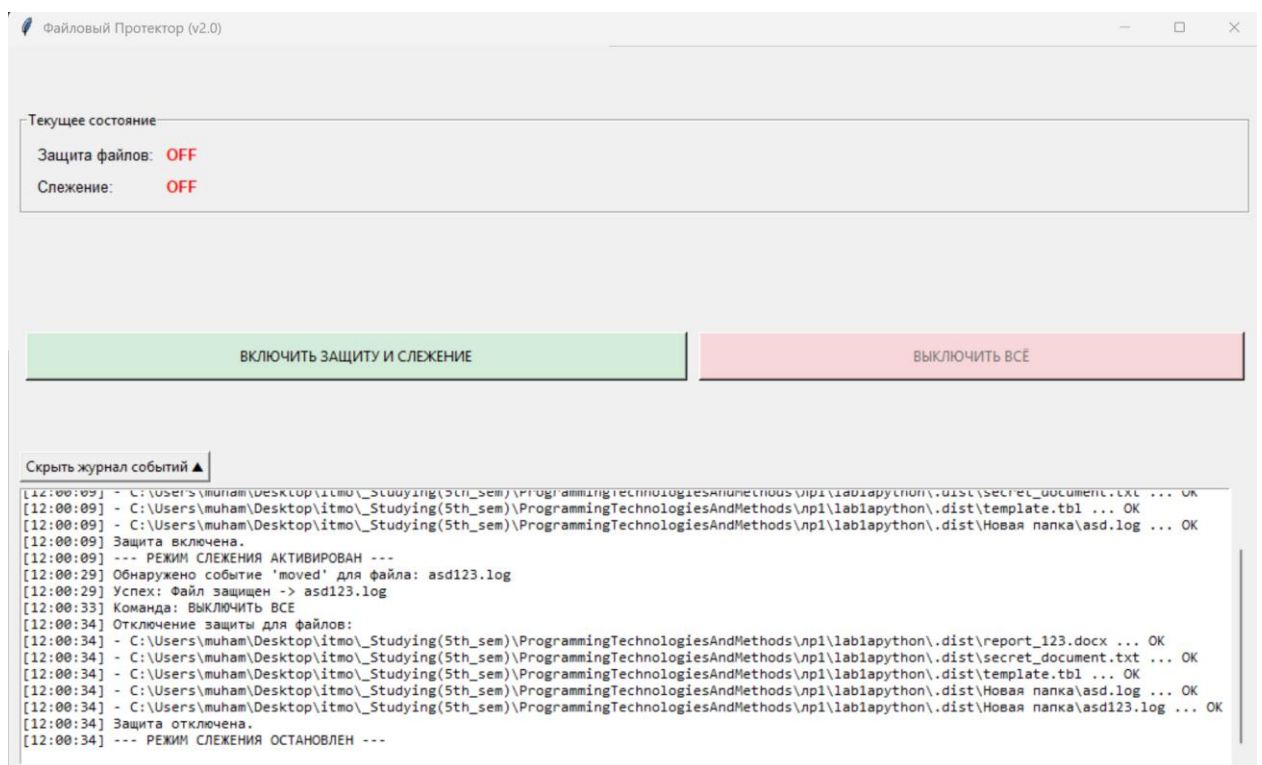


Рисунок 1 – Пример работы программы

2 ЗАДАНИЕ 1(Б): ВЕБ-СКРИПТ

2.1 Задание

Реализовать JavaScript, встраиваемый в HTML-документ, для защиты контента от копирования (выделение, буфер обмена, скриншоты) и сохранения. При этом должна сохраняться возможность печати страницы. Отключение скрипта должно выполняться по паролю, хеш которого хранится в самом скрипте.

2.2 Методы защиты контента на веб-страницах

Защита контента на стороне клиента (в браузере) является комплексной задачей и реализуется комбинацией технологий:

- **CSS (Cascading Style Sheets):** Свойство `user-select: none;` запрещает пользователю выделять текст на странице.
- **JavaScript (DOM Events):** Перехват событий мыши и клавиатуры позволяет блокировать стандартные действия. Например, отмена действия по умолчанию для события `contextmenu` блокирует вызов контекстного меню (правый клик), а для `keydown` – комбинации клавиш (Ctrl+C, Ctrl+S).
- **Web Crypto API:** Современный стандарт для выполнения криптографических операций в браузере. Используется для безопасного сравнения паролей: введенный пользователем пароль хешируется с помощью `crypto.subtle.digest()` и сравнивается с эталонным хешем, хранящимся в коде.
- **CSS Media Queries:** Правило `@media print` позволяет применять стили, которые активны только при печати документа. Это используется для скрытия защитных элементов (например, водяных знаков) с печатной версии страницы.

2.3 Практическая реализация

Скрипт реализован на чистом JavaScript (ES6+) и не требует внешних библиотек. Он встраивается в HTML-документ и активируется при загрузке страницы.

Реализованные механизмы защиты:

- **Блокировка выделения и копирования:** С помощью CSS-свойства `user-select: none` и перехвата событий `contextmenu` (правый клик) и `keydown` (для комбинаций Ctrl+C, Ctrl+U, Ctrl+S, F12) блокируются основные способы копирования текста и исходного кода.

- **Защита от скриншотов:** При бездействии пользователя более 500 мс на экране появляется полупрозрачный слой с предупреждением. Любое движение мыши или скроллинг сбрасывает таймер и временно скрывает этот слой, мешая сделать «чистый» скриншот.
- **Защита от сохранения страницы:** При загрузке скрипт проверяет протокол `location.protocol`. Если страница открыта как локальный файл (`file:`), содержимое скрывается, что делает сохраненную копию бесполезной.
- **Возможность печати:** CSS-правило `@media print` скрывает все защитные элементы (водяной знак) при отправке страницы на печать.
- **Отключение по паролю:** Комбинация клавиш `Ctrl + Alt + D` вызывает диалоговое окно для ввода пароля. Введенное значение хешируется с помощью асинхронной функции `crypto.subtle.digest('SHA-256', ...)` и сравнивается с эталонным хешем `P_HASH`. В случае успеха все обработчики событий удаляются, и защита отключается.

2.4 Демонстрация работы

Скрипт был развернут для демонстрации по адресу: <https://jk12thebest.github.io/lab1b/>. Пароль для отключения защиты: 123.

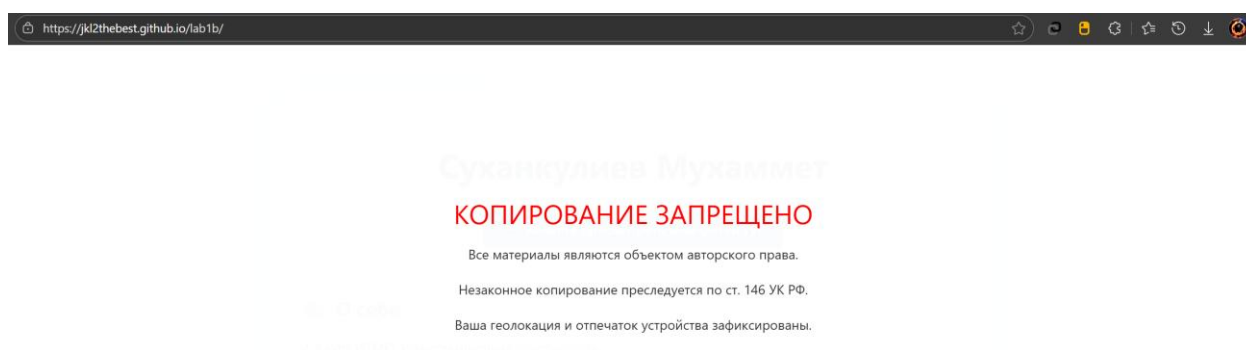


Рисунок 1 – Попытка сделать скриншот

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были успешно разработаны и протестированы два программных продукта, решающих поставленные задачи по защите информации.

Локальная программа «Файловый Протектор» продемонстрировала эффективность использования системных механизмов Windows (DACL) для надежной блокировки доступа к файлам. Реализация двойного интерфейса (CLI и GUI) и режима мониторинга в реальном времени делает ее гибким инструментом.

Веб-скрипт показал комплексный подход к защите контента на стороне клиента, сочетая в себе блокировку действий пользователя через CSS и JavaScript, противодействие созданию скриншотов и безопасную аутентификацию по паролю с использованием Web Crypto API.

Это позволило получить и закрепить навыки работы с механизмами безопасности ОС, управления правами доступа, хеширования и хранения паролей, а также применения средств HTML, CSS и JavaScript для защиты клиентского контента.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ищенко А. П. Технологии и методы программирования – Лекция 1: материалы лекции / А. П. Ищенко. – Санкт-Петербург : Университет ИТМО, 2025. – Текст : непосредственный.
2. Ищенко А. П. Технологии программирования: жизненный цикл и разработка ПО – Лекция 2: материалы лекции / А. П. Ищенко. – Санкт-Петербург : Университет ИТМО, 2025. – Текст : непосредственный.
3. Функция SetNamedSecurityInfoA (aclapi.h) : техническая документация. – Текст : электронный. – 2024. – URL: <https://learn.microsoft.com/ru-ru/windows/win32/api/aclapi/nf-aclapi-setnamedsecurityinfoa>
4. Функция SetFileSecurityA (winbase.h) : техническая документация. – Текст : электронный. – 2023. – URL: <https://learn.microsoft.com/ru-ru/windows/win32/api/winbase/nf-winbase-setfilesecuritya>
5. Русинович М. Внутреннее устройство Microsoft Windows / М. Русинович, Д. Соломон, А. Ионеску. – 6-е изд. – Санкт-Петербург : Питер, 2013. – 800 с. – Текст : непосредственный.
6. Библиотека watchdog для мониторинга событий файловой системы : документация библиотеки. – Текст : электронный. – 2023. – URL: <https://pypi.org/project/watchdog/>
7. Флэнаган Д. JavaScript. Подробное руководство / Д. Флэнаган. – 7-е изд. – O'Reilly Media, 2020. – 704 с. – Текст : непосредственный.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг А.1 – protector.py

```
import os
import sys
import glob
import time
import hashlib
import ctypes
import base64
import subprocess
import fnmatch
from threading import Thread
from ctypes import import_wintypes

# --- Константы ---
APP_NAME = "Файловый Протектор"
APP_VERSION = "2.0"
TEMPLATE_FILE = "template.tbl"

# --- Отложенные импорты ---
_pywintypes, _win32security, _win32con = None, None, None
_getpass, _argparse = None, None
_Observer, _FileSystemEventHandler = None, None
_tk, _simplifiedialog, _scrolledtext, _messagebox = None, None, None, None

def _import_win_deps():
    """Импорты Windows API."""
    global _pywintypes, _win32security, _win32con
    if _win32security is None:
        import pywintypes as _pywintypes
        import win32security as _win32security
        import win32con as _win32con

def _import_cli_deps():
    """Импорты для CLI."""
    global _getpass, _argparse
    if _argparse is None:
        import getpass as _getpass
        import argparse as _argparse

def _import_watcher_deps():
    """Импорты для режима слежения."""
    global _Observer, _FileSystemEventHandler
    if _Observer is None:
        from watchdog.observers import Observer as _Observer
        from watchdog.events import FileSystemEventHandler as
        _FileSystemEventHandler

def _import_gui_deps():
    """Импорты для GUI."""
    global _tk, _simplifiedialog, _scrolledtext, _messagebox
    if _tk is None:
```

```

import tkinter as _tk
from tkinter import simpledialog as _simplifiedialog
from tkinter import scrolledtext as _scrolledtext
from tkinter import messagebox as _messagebox

# === Основная логика и классы ===
def configure_utf8_stdout():
    """stdout/stderr на UTF-8. (Кириллица)"""
    if sys.stdout and sys.stdout.encoding != "utf-8":
        sys.stdout.reconfigure(encoding="utf-8")
    if sys.stderr and sys.stderr.encoding != "utf-8":
        sys.stderr.reconfigure(encoding="utf-8")

def log_cli(message):
    print(message, flush=True)

class ConfigManager:
    """Модуль управления чтением, записью и проверкой файла конфигурации."""

    def __init__(self, file_path):
        self.path = os.path.abspath(file_path)
        self.salt, self.password_hash, self.patterns = None, None, []

    def exists(self):
        return os.path.exists(self.path)

    def load(self):
        try:
            with open(self.path, "r", encoding="utf-8") as f:
                header = f.readline().strip()
                if ":" in header:
                    self.salt, self.password_hash = header.split(":", 1)
                self.patterns = [
                    line.strip()
                    for line in f
                    if line.strip() and not line.strip().startswith("#")
                ]
            return True
        except (IOError, ValueError):
            return False

    def create(self, password):
        salt_bytes = os.urandom(16)
        self.salt = base64.b64encode(salt_bytes).decode("utf-8")
        self.password_hash = self._hash_password(password, self.salt)
        with open(self.path, "w", encoding="utf-8") as f:
            f.write(f"{self.salt}:{self.password_hash}\n\n")
            f.write("# --- Шаблоны защищаемых файлов ---\n")
            f.write(
                "# Каждая строка - это маска файла (например, *.docx, "
                f"report_?.doc, secret.txt)\n"
            )
            f.write("# Пустые строки и строки, начинающиеся с #, "
                    f"игнорируются.\n\n")
            f.write(f"secret_document.txt\nreport_*.docx\n*.log\n")

    def verify_password(self, password):
        if not self.salt or not self.password_hash:
            return False

```

```

        return self._hash_password(password, self.salt) == self.password_hash

    @staticmethod
    def _hash_password(password, salt):
        return hashlib.sha256(
            salt.encode("utf-8") + password.encode("utf-8")
        ).hexdigest()

class SecurityManager:
    """Управляет правами доступа к файлам через Windows DACL."""

    def __init__(self):
        _import_win_deps()
        self.everyone_sid = _win32security.CreateWellKnownSid(
            _win32security.WinWorldSid
        )

    def set_deny_all(self, file_path, enable):
        if not os.path.exists(file_path):
            return False, f"Файл не найден: {file_path}"
        try:
            sd = _win32security.GetNamedSecurityInfo(
                file_path,
                _win32security.SE_FILE_OBJECT,
                _win32security.DACL_SECURITY_INFORMATION,
            )
            dacl = sd.GetSecurityDescriptorDacl() or _win32security.ACL()
            aces_to_remove = [
                i
                for i in range(dacl.GetAceCount())
                if dacl.GetAce(i)[2] == self.everyone_sid
                and dacl.GetAce(i)[0][0] ==
_win32security.ACCESS_DENIED_ACE_TYPE
            ]
            for i in sorted(aces_to_remove, reverse=True):
                dacl.DeleteAce(i)
            if enable:
                dacl.AddAccessDeniedAceEx(
                    _win32security.ACL_REVISION,
                    0,
                    _win32con.GENERIC_ALL,
                    self.everyone_sid,
                )
            _win32security.SetNamedSecurityInfo(
                file_path,
                _win32security.SE_FILE_OBJECT,
                _win32security.DACL_SECURITY_INFORMATION,
                None,
                None,
                dacl,
                None,
            )
            return True, f"Файл защищен -> {os.path.basename(file_path)}"
        except _pywintypes.error as e:
            return (
                False,
                f"Ошибка изменения прав для {os.path.basename(file_path)}:
{e}",
            )

    def check_protection_status(self, file_path):

```

```

    if not os.path.exists(file_path):
        return "Не настроено"
    try:
        with open(file_path, "r", encoding="utf-8"):
            pass
        return "OFF"
    except IOError:
        return "ON"

class FileWatcher:
    """Слежение за ФС в отдельном потоке (watchdog)."""

    def __init__(self, patterns, security_manager, logger_callback):
        _import_watcher_deps()
        self.patterns = patterns
        self.security_manager = security_manager
        self.logger = logger_callback
        self.observer = _Observer()
        self.handler = self._create_handler()
        self._is_running = False

    def _create_handler(self):
        class WatcherHandler(_FileSystemEventHandler):
            def __init__(self, patterns, sec_manager, logger):
                self.patterns = patterns
                self.sec_manager = sec_manager
                self.logger = logger

            def process(self, event):
                if event.is_directory:
                    return
                path = getattr(event, "dest_path", event.src_path)
                filename = os.path.basename(path)
                # ИСПРАВЛЕНИЕ 1: Используем fnmatch.fnmatch вместо
                несуществующего glob.fnmatch.fnmatch
                if any(fnmatch.fnmatch(filename, p) for p in self.patterns):
                    self.logger(
                        f"Обнаружено событие '{event.event_type}' для файла:
                        {filename}")
                    )
                    time.sleep(0.5)
                    success, msg = self.sec_manager.set_deny_all(path,
                    enable=True)
                    self.logger(f"{'Успех' if success else 'Ошибка'}: {msg}")

            def on_created(self, event):
                self.process(event)

            def on_moved(self, event):
                self.process(event)

        return WatcherHandler(self.patterns, self.security_manager,
        self.logger)

    def start(self):
        self._is_running = True
        self.observer.schedule(self.handler, path=".", recursive=True)
        self.observer.start()
        self.logger("--- РЕЖИМ СЛЕЖЕНИЯ АКТИВИРОВАН ---")

    def stop(self):

```

```

        if self._is_running:
            self.observer.stop()
            self.observer.join()
            self._is_running = False
            self.logger("--- РЕЖИМ СЛЕЖЕНИЯ ОСТАНОВЛЕН ---")

# --- Утилитарные и платформо-зависимые функции ---
def find_target_files(patterns, include_config_path):
    target_files = {
        os.path.abspath(f)
        for p in patterns
        for f in glob.glob(f"**/{p}", recursive=True)
        if os.path.isfile(f)
    }
    target_files.add(include_config_path)

# ИСПРАВЛЕНИЕ 2: Исключаем и .exe, и .py файл для надежности
if getattr(sys, "frozen", False):
    # Если скомпилировано, sys.executable - это путь к .exe
    self_executable_path = os.path.abspath(sys.executable)
    target_files.discard(self_executable_path)
else:
    # Если запускается как скрипт, __file__ - это путь к .py
    self_script_path = os.path.abspath(__file__)
    target_files.discard(self_script_path)

    return sorted(list(target_files))

def is_admin():
    try:
        return ctypes.windll.shell32.IsUserAnAdmin() != 0
    except Exception:
        return False

class SHELLEXECUTEINFO(ctypes.Structure):
    """Структура данных для вызова WinAPI ShellExecuteExW."""

    _fields_ = [
        ("cbSize", ctypes.DWORD),
        ("fMask", ctypes.c_ulong),
        ("hwnd", ctypes.HWND),
        ("lpVerb", ctypes.LPCWSTR),
        ("lpFile", ctypes.LPCWSTR),
        ("lpParameters", ctypes.LPCWSTR),
        ("lpDirectory", ctypes.LPCWSTR),
        ("nShow", ctypes.c_int),
        ("hInstApp", ctypes.HINSTANCE),
        ("lpIDList", ctypes.c_void_p),
        ("lpClass", ctypes.LPCWSTR),
        ("hkeyClass", ctypes.HKEY),
        ("dwHotKey", ctypes.DWORD),
        ("hIcon", ctypes.HANDLE),
        ("hProcess", ctypes.HANDLE),
    ]

def elevate_privileges_cli():
    """
    Перезапускает приложение с правами администратора для CLI.

```

```

"""
SEE_MASK_NOCLOSEPROCESS = 0x00000040
# Описание структуры SHELLEXECUTEINFO (Иначе ошибки Access Violation).
info = SHELLEXECUTEINFO(
    cbSize=ctypes.sizeof(SHELLEXECUTEINFO),
    fMask=SEE_MASK_NOCLOSEPROCESS,
    lpVerb="runas",
    lpFile=sys.executable,
    lpParameters=" ".join(f'"{arg}"' for arg in sys.argv[1:]),
    nShow=1,
)

try:
    # Запуск с повышенными правами
    if not ctypes.windll.shell32.ShellExecuteExW(ctypes.byref(info)):
        # Если пользователь нажал "Нет" в UAC, ShellExecuteExW вернет
False
        # и GetLastError() вернет ERROR_CANCELLED (1223)
        error_code = ctypes.windll.kernel32.GetLastError()
        if error_code == 1223:
            print("Операция отменена пользователем.")
        else:
            raise ctypes.WinError()
        sys.exit(1)

    hProcess = info.hProcess

    # Ожидаем завершения дочернего процесса, но позволяем прервать
ожидание
    try:
        ctypes.windll.kernel32.WaitForSingleObject(hProcess, -1)
    except KeyboardInterrupt:
        # Процесс-потомок продолжит работать, но родитель завершится.
        print("\nОжидание дочернего процесса прервано.")

    ctypes.windll.kernel32.CloseHandle(hProcess)

except KeyboardInterrupt:
    # Ctrl+C до или во время вызова ShellExecuteExW
    print("\nОперация прервана пользователем.")
except Exception as e:
    print(f"Ошибка при запросе прав администратора: {e}")

# Завершение текущего (не-админского) процесса
sys.exit(0)

def hide_console_window():
    """Скрывает окно консоли, привязанное к текущему процессу."""
    whnd = ctypes.windll.kernel32.GetConsoleWindow()
    if whnd != 0:
        ctypes.windll.user32.ShowWindow(whnd, 0)

# === Классы-контроллеры и GUI ===
class AppController:
    """Координирует работу всех модулей."""

    def __init__(self, logger_callback=log_cli):
        self.config, self.security, self.watcher, self.log = (
            ConfigManager(TEMPLATE_FILE),
            SecurityManager(),

```

```

        None,
        logger_callback,
    )

def setup(self):
    _import_cli_deps()
    self.log("---- Первоначальная настройка ----")

    # Существует ли файл и включена ли защита.
    protection_was_on = self.get_status() == "ON"
    if self.config.exists():
        if self.log == log_cli:
            confirm = input(
                f"'{TEMPLATE_FILE}' уже существует. Перезаписать? (y/n): "
                ).lower()
            if confirm != "y":
                self.log("Отмена.")
                return False
        if protection_was_on:
            self.log("\nВНИМАНИЕ: Файл конфигурации в данный момент
защищен.")
            self.log("Вы можете полностью сбросить текущие настройки.")

            confirmation_code = input(
                "Для подтверждения сброса введите код подтверждения: "
            )
            if confirmation_code != "12345": # Код подтверждения сброса
настроек
                self.log("Неверный код подтверждения. Отмена
перенастройки.")
                return False

            self.log("Код подтвержден.")
            self.security.set_deny_all(self.config.path, enable=False)

        if self.log == log_cli:
            while True:
                password = _getpass.getpass("Введите новый пароль: ")
                if not password:
                    self.log("Пароль не может быть пустым.")
                    continue
                if password == _getpass.getpass("Подтвердите пароль: "):
                    break
                self.log("Пароли не совпадают.")
            self.config.create(password)
            self.log(f"Файл '{TEMPLATE_FILE}' успешно создан с инструкциями
внутри.")
            return True

def get_status(self):
    return self.security.check_protection_status(self.config.path)

def protect_on(self):
    if self.get_status() == "ON":
        self.log("Защита уже включена.")
        return False, []
    if not self.config.load():
        self.log("Не удалось загрузить конфигурацию.")
        return False, []
    patterns_to_watch = self.config.patterns

```

```

        target_files = find_target_files(self.config.patterns,
self.config.path)
        self.log("Включение защиты для файлов:")
        for f in target_files:
            success, _ = self.security.set_deny_all(f, enable=True)
            self.log(f" - {f} ... {'OK' if success else 'ОШИБКА'}")
        self.log("\nЗащита включена.")
        return True, patterns_to_watch

    def protect_off(self, password=None):
        if self.get_status() == "OFF":
            self.log("Защита уже выключена.")
            return {"status": "no_change"}
        if password is None:
            _import_cli_deps()
            password = _getpass.getpass("Введите пароль: ")
        self.security.set_deny_all(self.config.path, enable=False)
        if not self.config.load() or not
self.config.verify_password(password):
            self.security.set_deny_all(self.config.path, enable=True)
            self.log("Неверный пароль! Файлы защищены.")
            return {
                "status": "error",
                "message": "Неверный пароль",
            }
        target_files = find_target_files(self.config.patterns,
self.config.path)
        self.log("Отключение защиты для файлов:")
        for f in target_files:
            success, _ = self.security.set_deny_all(f, enable=False)
            self.log(f" - {f} ... {'OK' if success else 'ОШИБКА'}")
        self.log("\nЗащита отключена.")
        return {"status": "success"}

    def start_watch(self, patterns):
        if self.watcher and self.watcher._is_running:
            self.log("Слежение уже запущено.")
            return
        if not patterns:
            self.log("Нет шаблонов для слежения.")
            return
        self.watcher = FileWatcher(patterns, self.security, self.log)
        self.watcher.start()

    def stop_watch(self):
        if self.watcher and self.watcher._is_running:
            self.watcher.stop()
        else:
            self.log("Слежение не было запущено.")

class ProtectorGUI(Thread):
    """Главный класс GUI, работающий в отдельном потоке."""

    def __init__(self):
        _import_gui_deps()
        super().__init__()
        self.daemon = True
        self.root = None
        self.controller = AppController(logger_callback=self.log)

    def run(self):

```

```

self.root = _tk.Tk()
self.root.title(f"{APP_NAME} (v{APP_VERSION})")
self.root.geometry("500x250")
self.root.minsize(500, 250)
if not self.controller.config.exists():
    _messagebox.showerror(
        "Программа не настроена", f"Файл '{TEMPLATE_FILE}' не
найден."
    )
    self.root.destroy()
    return
self._create_widgets()
self.update_all_statuses()
self.root.protocol("WM_DELETE_WINDOW", self.on_closing)
self.root.mainloop()

def _create_widgets(self):
    main_frame = _tk.Frame(self.root, padx=10, pady=10)
    main_frame.pack(fill="both", expand=True)
    status_frame = _tk.LabelFrame(
        main_frame, text="Текущее состояние", padx=10, pady=10
    )
    status_frame.pack(fill="x", expand=True)
    _tk.Label(status_frame, text="Защита файлов:", font=("Arial",
10)).grid(
        row=0, column=0, sticky="w"
    )
    self.status_value = _tk.Label(
        status_frame, text="...", font=("Arial", 10, "bold")
    )
    self.status_value.grid(row=0, column=1, sticky="w", padx=5)
    _tk.Label(status_frame, text="Слежение:", font=("Arial", 10)).grid(
        row=1, column=0, sticky="w", pady=(5, 0)
    )
    self.watch_status_value = _tk.Label(
        status_frame, text="...", font=("Arial", 10, "bold")
    )
    self.watch_status_value.grid(row=1, column=1, sticky="w", padx=5,
pady=(5, 0))
    button_frame = _tk.Frame(main_frame, pady=10)
    button_frame.pack(fill="x", expand=True)
    self.on_button = _tk.Button(
        button_frame,
        text="ВКЛЮЧИТЬ ЗАЩИТУ И СЛЕЖЕНИЕ",
        command=self.enable_all,
        bg="#D4EDDA",
        height=2,
    )
    self.on_button.pack(side="left", expand=True, fill="x", padx=5)
    self.off_button = _tk.Button(
        button_frame,
        text="ВЫКЛЮЧИТЬ ВСЁ",
        command=self.disable_all,
        bg="#F8D7DA",
        height=2,
    )
    self.off_button.pack(side="left", expand=True, fill="x", padx=5)
    log_frame = _tk.Frame(main_frame)
    log_frame.pack(fill="both", expand=True, pady=(5, 0))
    self.log_button = _tk.Button(
        log_frame,
        text="Открыть журнал событий ▼",

```

```

        command=self.toggle_logs,
    )
    self.log_button.pack(anchor="w")
    self.log_box = _scrolledtext.ScrolledText(
        log_frame, height=10, state="disabled", font=("Consolas", 9),
wrap=_tk.WORD
    )

    def log(self, message):
        def _log_thread_safe():
            if self.log_box.winfo_exists():
                self.log_box.config(state="normal")
                timestamp = time.strftime("%H:%M:%S")
                self.log_box.insert(_tk.END, f"[{timestamp}]
{message.strip()}\n")
                self.log_box.see(_tk.END)
                self.log_box.config(state="disabled")

        if self.root and self.log_box.winfo_exists():
            self.root.after(0, _log_thread_safe)

    def update_all_statuses(self):
        file_status = self.controller.get_status()
        watch_status = (
            "ON"
            if self.controller.watcher and
self.controller.watcher._is_running
            else "OFF"
        )
        colors = {"ON": "green", "OFF": "red", "Не настроено": "orange"}
        watch_colors = {"ON": "green", "OFF": "red"}
        self.status_value.config(text=file_status, fg=colors.get(file_status,
"black"))
        self.watch_status_value.config(
            text=watch_status, fg=watch_colors.get(watch_status, "black")
        )
        is_active = file_status == "ON" or watch_status == "ВКЛЮЧЕНО"
        self.on_button.config(state="disabled" if is_active else "normal")
        self.off_button.config(state="normal" if is_active else "disabled")

    def enable_all(self):
        self.log("Команда: ВКЛЮЧИТЬ ВСЕ")
        Thread(target=self._enable_all_task, daemon=True).start()

    def _enable_all_task(self):
        success, patterns = self.controller.protect_on()
        if success:
            self.controller.start_watch(patterns)
            self.root.after(100, self.update_all_statuses)

    def disable_all(self):
        self.log("Команда: ВЫКЛЮЧИТЬ ВСЕ")
        password = _simplifiedialog.askstring(
            "Требуется пароль",
            "Введите пароль, чтобы отключить защиту файлов:",
            show="*",
            parent=self.root,
        )
        if password is not None:
            Thread(target=self._disable_all_task, args=(password,),
daemon=True).start()
        else:

```

```

        self.log("Отключение отменено.")

def _disable_all_task(self, password):
    result = self.controller.protect_off(password)
    if result.get("status") == "success":
        self.controller.stop_watch()
    self.root.after(100, self.update_all_statuses)

def toggle_logs(self):
    if self.log_box.wininfo_viewable():
        self.log_box.pack_forget()
        self.root.geometry("500x250")
        self.log_button.config(text="Открыть журнал событий ▼")
    else:
        self.root.geometry("500x450")
        self.log_box.pack(fill="both", expand=True, pady=(5, 0))
        self.log_button.config(text="Скрыть журнал событий ▲")

def on_closing(self):
    self.log("Завершение работы, остановка слежения...")
    self.root.withdraw()
    self.controller.stop_watch()
    self.root.destroy()

# === Точка входа ===
def main():
    """Главная функция, определяющая режим работы приложения (GUI или
CLI)."""
    # Установка рабочего каталога
    if getattr(sys, "frozen", False):
        os.chdir(os.path.dirname(sys.executable))

    # Справка не требует прав администратора.
    if "--help" in sys.argv or "-h" in sys.argv:
        _import_cli_deps()
        configure_utf8_stdout()
        parser = _argparse.ArgumentParser(
            prog=os.path.basename(sys.executable),
            description=f"{APP_NAME} (v{APP_VERSION}) - Утилита для защиты
файлов.",
        )
        subparsers = parser.add_subparsers(dest="command", title="Доступные
команды")
        subparsers.add_parser(
            "setup", help="Первоначальная настройка и создание " +
TEMPLATE_FILE
        )
        subparsers.add_parser("status", help="Проверить текущий статус
защиты.")
        subparsers.add_parser(
            "on", help="Включить защиту для файлов из " + TEMPLATE_FILE
        )
        subparsers.add_parser("off", help="Отключить защиту (требуется
пароль).")
        subparsers.add_parser(
            "watch", help="Включить режим постоянного слежения и защиты."
        )
        parser.print_help()
        input("\nНажмите Enter для выхода...")
        return

```

```

# CLI, если есть аргументы (кроме служебного).
is_cli_mode = len(sys.argv) > 1 and "--run-gui" not in sys.argv
if is_cli_mode:
    # --- Режим командной строки ---
    _import_cli_deps()
    configure_utf8_stdout()
    if not is_admin():
        elevate_privileges_cli()

    parser =
_argparse.ArgumentParser(prog=os.path.basename(sys.executable))
    subparsers = parser.add_subparsers(dest="command", required=True)
    subparsers.add_parser("setup")
    subparsers.add_parser("status")
    subparsers.add_parser("on")
    subparsers.add_parser("off")
    subparsers.add_parser("watch")

    try:
        args = parser.parse_args()
    except SystemExit:
        input("\nНажмите Enter для выхода...")
        return

    controller = AppController()
    if args.command != "setup" and not controller.config.exists():
        print("Программа не настроена. Сначала выполните команду
'setup'.")
    else:
        if args.command == "setup":
            controller.setup()
        elif args.command == "status":
            print(f"Статус защиты: {controller.get_status()}")
        elif args.command == "on":
            controller.protect_on()
        elif args.command == "off":
            controller.protect_off()
        elif args.command == "watch":
            if controller.config.load():
                controller.start_watch(controller.config.patterns)
            try:
                while True:
                    time.sleep(1)
            except KeyboardInterrupt:
                controller.stop_watch()
        else:
            print("Не удалось загрузить конфигурацию для слежения.")

    input("\nНажмите Enter для завершения...")

else:
    # Запуск из консоли уже с правами.
    if is_admin() and "--run-gui" not in sys.argv:
        hide_console_window()
        gui_app = ProtectorGUI()
        gui_app.start()
        gui_app.join()
        return

    # Обычный двойной клик (без прав, без --run-gui).
    if not is_admin() and "--run-gui" not in sys.argv:
        subprocess.Popen(

```

```

        [sys.executable, "--run-gui"],
        close_fds=True,
        creationflags=subprocess.DETACHED_PROCESS,
    )
    return

# Фоновый процесс-лаунчер (без прав, но с --run-gui).
if not is_admin() and "--run-gui" in sys.argv:
    try:
        result = ctypes.windll.shell32.ShellExecuteW(
            None, "runas", sys.executable, "--run-gui", None, 1
        )
        if result <= 32:
            _import_gui_deps()
            _tk.Tk().withdraw()
            _messagebox.showerror(
                "Ошибка прав",
                "Для работы программы требуются права
администратора.",
            )
        except Exception:
            pass
    return

# Финальный процесс (с правами и с --run-gui).
# Прячем консоль и запускаем GUI.
if is_admin() and "--run-gui" in sys.argv:
    hide_console_window()
    gui_app = ProtectorGUI()
    gui_app.start()
    gui_app.join()
    return

if __name__ == "__main__":
    main()

```

Листинг А.2 – script.js

```

document.addEventListener('DOMContentLoaded', () => {
    if (location.protocol === 'фыв:') {
        document.body.innerHTML = `<div style="text-align:center;margin-
top:2em;font-family:sans-serif;">Контент защищен!</div>`;
        return () => {};
    }

    const P_HASH =
'a665a45920422f9d417e4867efdc4fb8a04a1f3ffff1fa07e998e86f7f7a27ae3'; // '123'
    const TIMEOUT = 500;
    let timer;

    const styleEl = document.createElement('style');
    styleEl.textContent = `
/* Блокировка выделения текста */
.prot {
    -webkit-user-select: none;
    user-select: none;
}

#wm {
    position: fixed;

```

```

        inset: 0;
        z-index: 9999;
        background: rgba(255, 255, 255, 0.98);
        color: red;
        font-size: 2em;
        text-align: center;
        padding-top: 20vh;

        pointer-events: none;

        opacity: 0;
        transition: opacity 0.2s ease-in-out;
    }

    #wm.vis {
        opacity: 1;
    }

    @media print {
        #wm {
            display: none !important;
        }
    }
`;
document.head.append(styleEl);

const watermarkEl = document.createElement('div');
watermarkEl.id = 'wm';
watermarkEl.innerHTML = `КОПИРОВАНИЕ ЗАПРЕЩЕНО<br>
    <small style="font-size:0.5em;color:#333;">
    Все материалы являются объектом авторского права.<br>
    Незаконное копирование преследуется по ст. 146 УК РФ.<br>
    Ваша геолокация и отпечаток устройства зафиксированы.</small>`;

const sha256 = async (str) => {
    const buf = await crypto.subtle.digest('SHA-256', new
TextEncoder().encode(str));
    return Array.from(new Uint8Array(buf)).map(b =>
b.toString(16).padStart(2, '0')).join('');
};

const handlers = {
    resetTimer: () => {
        watermarkEl.classList.remove('vis');
        clearTimeout(timer);
        timer = setTimeout(() => watermarkEl.classList.add('vis'),
TIMEOUT);
    },
    keydown: async (e) => {
        handlers.resetTimer();
        if (e.ctrlKey && e.altKey && e.key === 'd') {
            const pass = prompt('Пароль для отключения защиты:');
            if (pass && await sha256(pass) === P_HASH) {
                toggleProtection(false);
            } else if (pass) {
                alert('Неверный пароль!');
            }
        }
        if ((e.ctrlKey && 'csu'.includes(e.key)) || e.key === 'F12' ||
(e.ctrlKey && e.shiftKey && e.key === 'I')) {
            e.preventDefault();
        }
    },
};

```

```

        prevent: (e) => e.preventDefault(),
    };

    const listeners = [
        ['mousemove', handlers.resetTimer],
        ['scroll', handlers.resetTimer],
        ['keydown', handlers.keydown],
        ['contextmenu', handlers.prevent]
    ];

    const toggleProtection = (enable) => {
        const method = enable ? 'addEventListener' : 'removeEventListener';
        listeners.forEach(([event, handler]) => document[method](event,
handler));

        document.body.classList.toggle('prot', enable);
        clearTimeout(timer);

        if (enable) {
            document.body.append(watermarkEl);
            handlers.resetTimer();
        } else {
            watermarkEl.remove();
            alert('Защита отключена.');
```